

1 Vision Transformer

1.1 Проблемы свёрточных нейронных сетей

Свёрточные нейронные сети до относительно недавнего времени были стандартом в задачах обработки изображений, каждый раз являясь основой для SOTA-архитектур. Однако, как и у многих подходов, у CNN есть свои проблемы.

Во-первых, это **отсутствие восприятия глобального контекста**. На каждом слое CNN обрабатывают изображения через локальные поля. Это означает, что они рассматривают только небольшую область за раз. Такой подход ограничивает возможность сети воспринимать глобальный контекст изображения. Например, CNN не могут гарантировать того, что информация из левого верхнего угла будет связана с информацией из правого нижнего. Либо, CNN будут связывать дальние области косвенно и постепенно, что плохо скажется на обучении сети.

Во-вторых, CNN используют **одинаковые фильтры**, применяя их ко всему изображению. Такой подход может быть неэффективен, если полезная информация распределена неравномерно.

Обсудим проблему отсутствия восприятия глобального контекста свёрточными сетями. Она кажется довольно знакомой: из-за локального восприятия контекста и последовательной обработки отдельных участков сети тяжело воспринимать всю информацию целиком. Да ведь это же почти та же самая проблема, которая была решена с помощью механизма внимания! А как мы уже выяснили, наиболее удачная архитектура, основанная на внимании — это трансформер.

Если задуматься, то трансформеры решают и вторую проблему: механизм самовнимания может отдавать приоритет различным участкам при неравномерно распределенной на изображении информации, а нацеленность архитектуры трансформера на работу с последовательностями может позволить работать с изображениями различного размера.

Однако, как мы можем адаптировать архитектуру трансформера, модели изначально предназначенной для решения задачи машинного перевода, для решения задач обработки изображений? Ведь, в отличие от предложений, состоящих из сравнительно небольшого количества элементов - токенов, изображения состоят из огромного количества пикселей. Даже небольшая картинка размером 256×256 состоит из 65536 пикселей. А если изображение будет больше? Тогда не хватит никакого контекстного окна, чтобы его обработать.

Посмотрим на проблему с другой стороны: а почему мы должны считать всего один пиксель — токеном? В трансформерах после embedding-слоя токен превращается в плотный вектор (например, размерностью 512). Будет довольно странно сравнивать с ним пиксель, который для трёхканальных изображений — трехмерный вектор. Что если попробовать подавать на вход трансформеру несколько пикселей изображения сразу, как бы нарезая его на несколько патчей? Примерно таким же вопросом в 2020м году задалась

авторы модели Vision Transformer (ViT).

1.2 Токенизация изображений

Авторы ViT решали задачу классификации изображений и решили никак не менять оригинальную архитектуру кодировщика трансформера, показав, что такой подход работает лучше всего. Различия с оригинальной статьёй заключаются лишь в способе обработки данных, поступающих на вход модели.

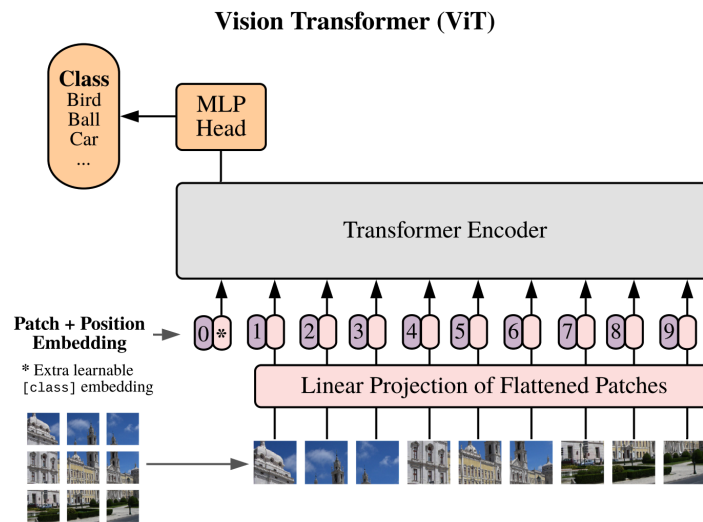


Рис. 1: Архитектура ViT. Взято из оригинальной статьи “An Image is Worth 16x16 Words”.

Для того чтобы подать изображение $x \in \mathbb{R}^{H \times W \times C}$ на кодировщик трансформера, авторы ViT предлагают разбивать его на патчи $16 \times 16 \times C$, потом проецировать в вектора размерностью D . Сделать это можно по-разному: можно выполнить с помощью линейного слоя, а можно с помощью свертки с размером ядра и шагом равными 16.

1.3 Обучаемое абсолютное позиционное кодирование

В отличие от текстовых последовательностей в задачах NLP, изображения имеют двумерную структуру. В оригинальной статье, посвященной модели трансформера, для решения проблемы позиционирования элементов последовательности использовалось одномерное позиционное кодирование. При обработке изображений логичным шагом будет применить двумерное позиционное кодирование для патчей. Авторы ViT сравнили одномерное и двумерное обучаемое позиционное кодирование и показали, что существенного

прироста точности для изображений фиксированного размера от использования двумерного метода нет.

1.4 CLS токен

Если обратить внимание на схему ViT, можно заметить токен под номером 0 — токен CLS. Концепция этого токена была впервые предложена в модели BERT, используемой для задач обработки естественного языка. CLS-токен добавляется в начало входной последовательности и служит для агрегации информации из всей последовательности, чтобы использовать её в задачах классификации. Его финальное состояние используется для агрегирования всей последовательности. Это позволяет избежать предвзятости, которая может возникнуть при выборе отдельного токена последовательности.